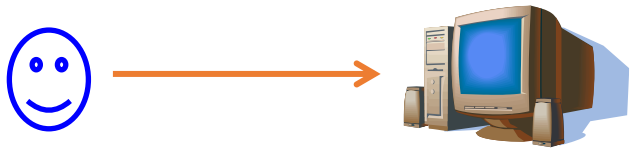
```
print("Computer Programming 2026\n")
print("Department of Geography\n")
print("5 March 2026\n")

print("Week 2")
```

Objectives

- Information and how to process information
- Communication between humans and calculators
- Algorithms
  - Pseudocode
  - Flowchart

Communication with the calculator

The user-programmer and the calculator must speak the same language



An option may be to “teach the calculator” to understand (= interpret and execute commands expressed in) a natural language such as English or Italian.

- Advantages:
  - Semantically-rich languages, able to express commands for sure
  - Languages already known by the user
- Disadvantages:
  - Semantically-rich languages, so at risk of ambiguity
  - Complex languages to teach / learn

An example

MOM: *“Alessandro, please, go to the store and buy 1 bottle of milk. If the store has eggs, buy 6”*

I got home with 6 bottles of milk

MOM: *“Why the hell did you buy 6 bottles of milk??”*

ME: “Because THE STORE HAS EGGS!!!”

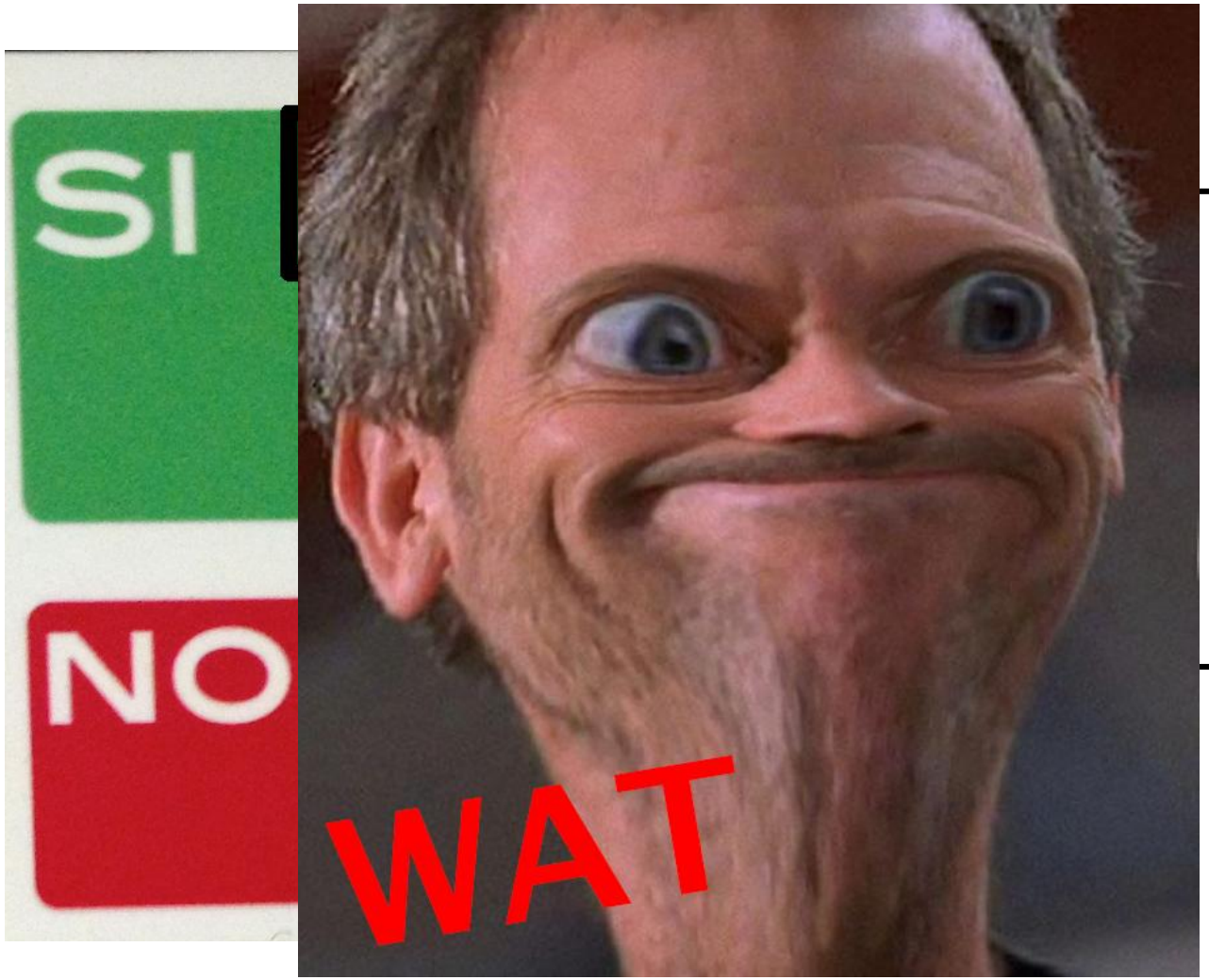
Natural Language

Natural Language: ambiguous!

BOTTLES, CONTAINERS AND PLASTIC BAGS

PLATES, CUPS AND OTHER PLASTIC OBJECTS

Natural Language: ambiguous!



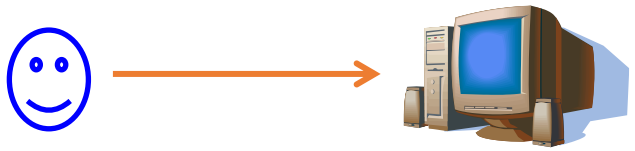
BOTTLES, CONTAINERS AND PLASTIC BAGS

PLATES, CUPS AND OTHER PLASTIC OBJECTS

HOW TO BE MORE FORMAL?

Communication through formal languages

The user-programmer and the calculator must speak the same language



An alternative option is to create a “programming” language, designated to the communication with the calculator.

- Advantages:
  - Specifically-designed language, so very efficient
  - Non-ambiguous language
- Disadvantages:
  - Formalized language, so structurally different from the natural languages
  - Language unknown to the user

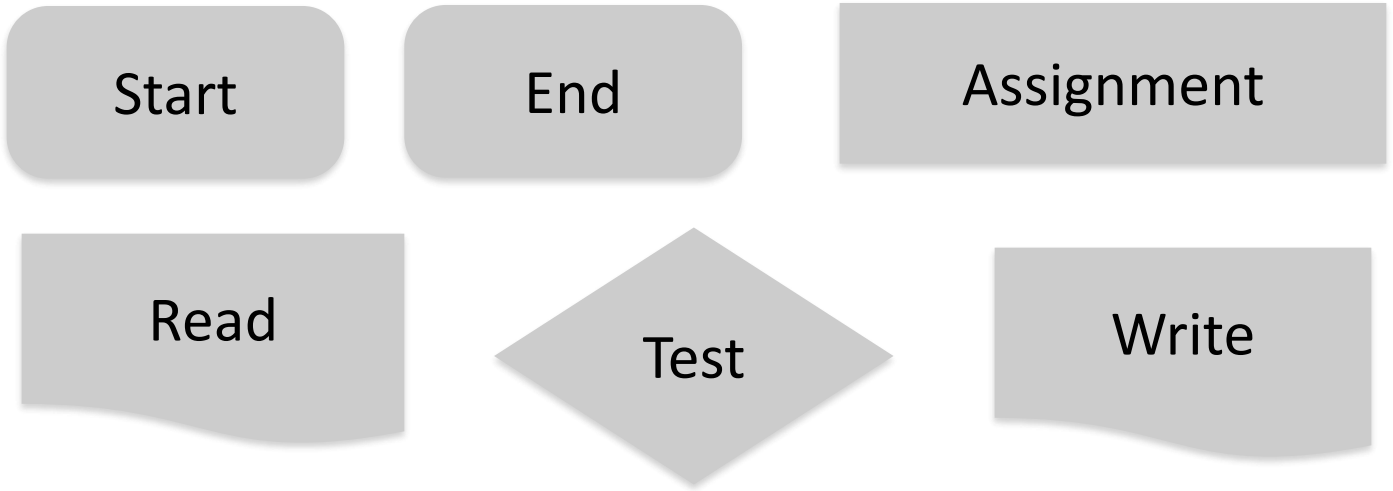
ALGORITHMS

How to specify an algorithm?

- Using pseudocode

If A > B **then** B = A **else** A = B

- Using flowchart



An example

MOM: *“Alessandro, please, go to the store and buy 1 bottle of milk. If the store has eggs, buy 6”*

An example

MOM: *"Alessandro, please, go to the store (A) AND buy 1 bottle of milk (B). IF the store has eggs (C), buy 6 (D)"*

An example

MOM: *"Alessandro, please, go to the store (A) AND buy 1 bottle of milk (B). IF the store has eggs (C), buy 6 (D)"*

PREDICATES:

A: go to the store
B: buy 1 bottle of milk
C: the store has eggs
D: buy 6

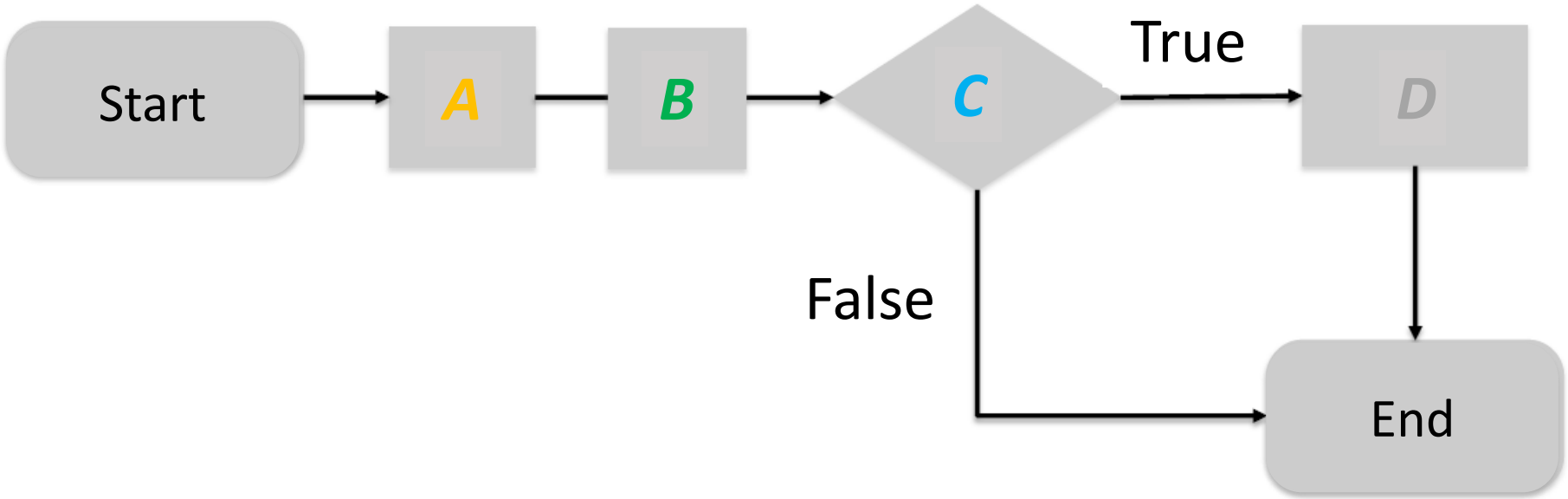
LOGICAL OPERATORS:

AND
IF

A and B
if C:
(then) D

An example

MOM: *"Alessandro, please, go to the store (A) AND buy 1 bottle of milk (B). IF the store has eggs (C), buy 6 (D)"*



HOW TO SUMMARIZE/ FORMALIZE ALL THIS?

Algorithm and Program

Algorithm

- (idea) Description of the solution of a problem, written in such a way to be able to be executed by an executor (potentially different from the author of the algorithm).
- (definition) Finite sequence of (non-ambiguous) atomic steps, which operate on data to reach the solution of a problem.

Program

- Algorithm written in such a way to be able to be executed by a calculator (automatic executor)

How to create an algorithm

- Part 0/4: The bad news!

How to create an algorithm

- Part 1/4: Understand the problem
  - What is the general problem that we are trying to solve?

How to create an algorithm

- Part 2/4: Design a plan
  - There may be different strategies to solve the same problem
    - Hypothesize and verify
    - Search for patterns
    - Solve smaller problems
    - Draw a diagram

How to create an algorithm

- Part 3/4: Carry out the plan
  - Put your plan into action!
  - Stay on the designed plan unless there are obvious reasons to believe it will not work anymore

Patience is your best ally

How to create an algorithm

- Part 4/4: Reason and comprehend
  - Comprehend what you have done and where the designed algorithm can be best applied

Practice is fundamental!

Test your work!!!

- If there is something that doesn't work in your program/algorithm, **users will find it**!

Greatest Common Divisor

Definition

- The Greatest Common Divisor (GCD) is the biggest among the divisors that are common to two or more numbers

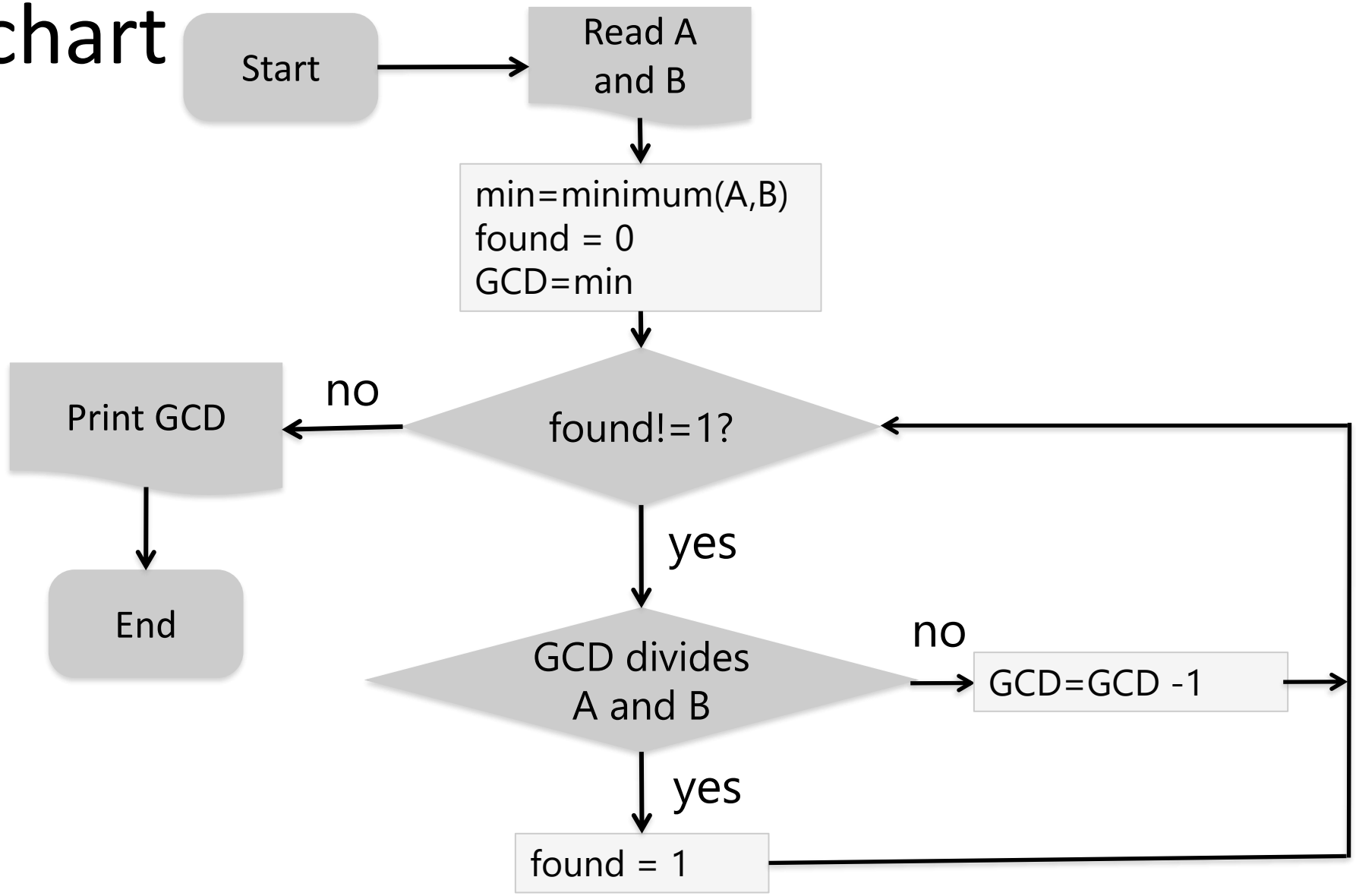
Examples

- Given: A=12, B=15
  - Common divisors: 1, 3
  - GCD=3
- Given: A=10, B=30 e C=20
  - Common divisors: 1, 2, 5, 10
  - GCD=10

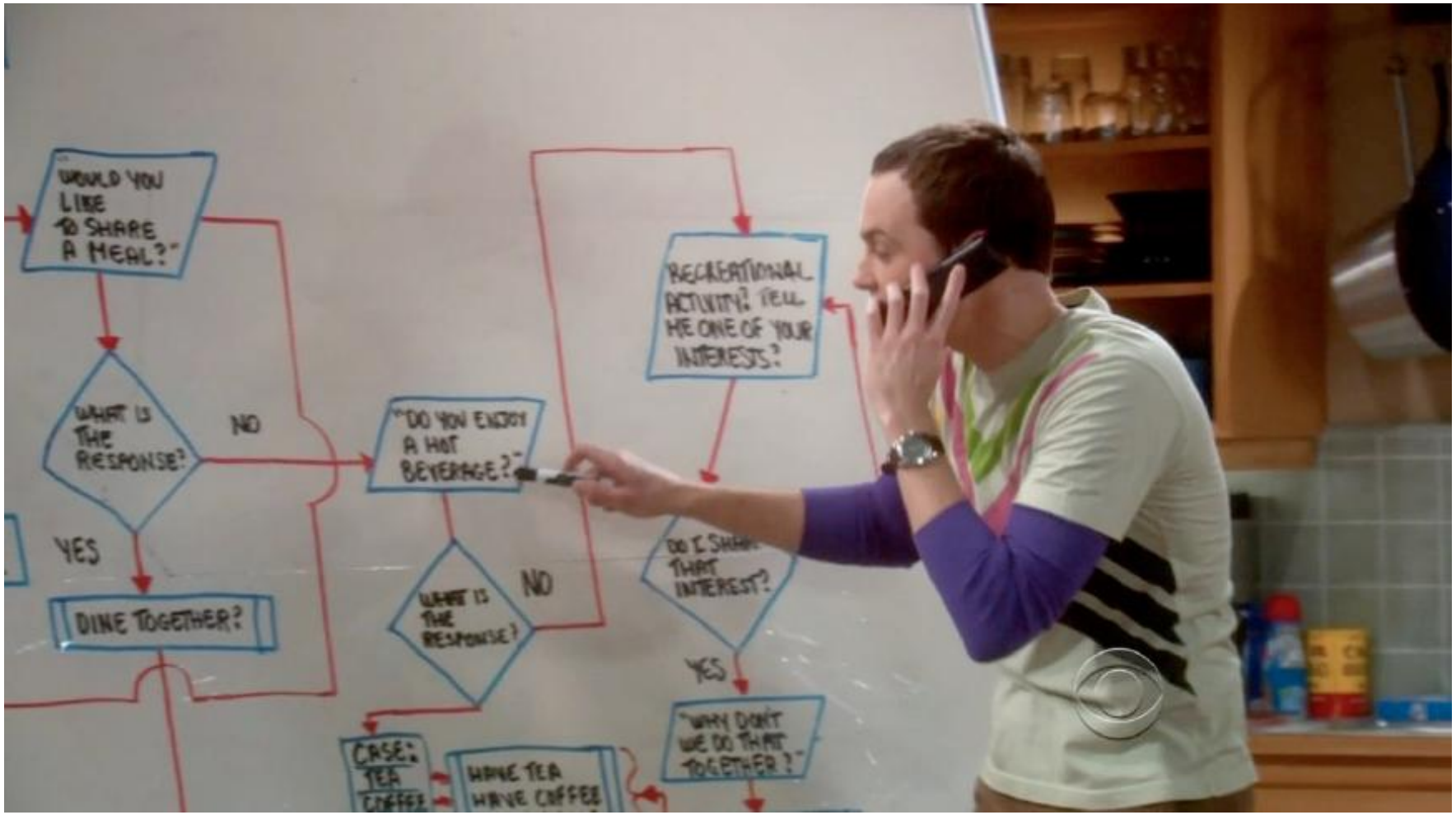
GCD: pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
   - **If** GCD divides A and B
     a. **Then** found = 1
     b. **Else** GCD = GCD - 1
5. **Print** GCD

GCD: flowchart

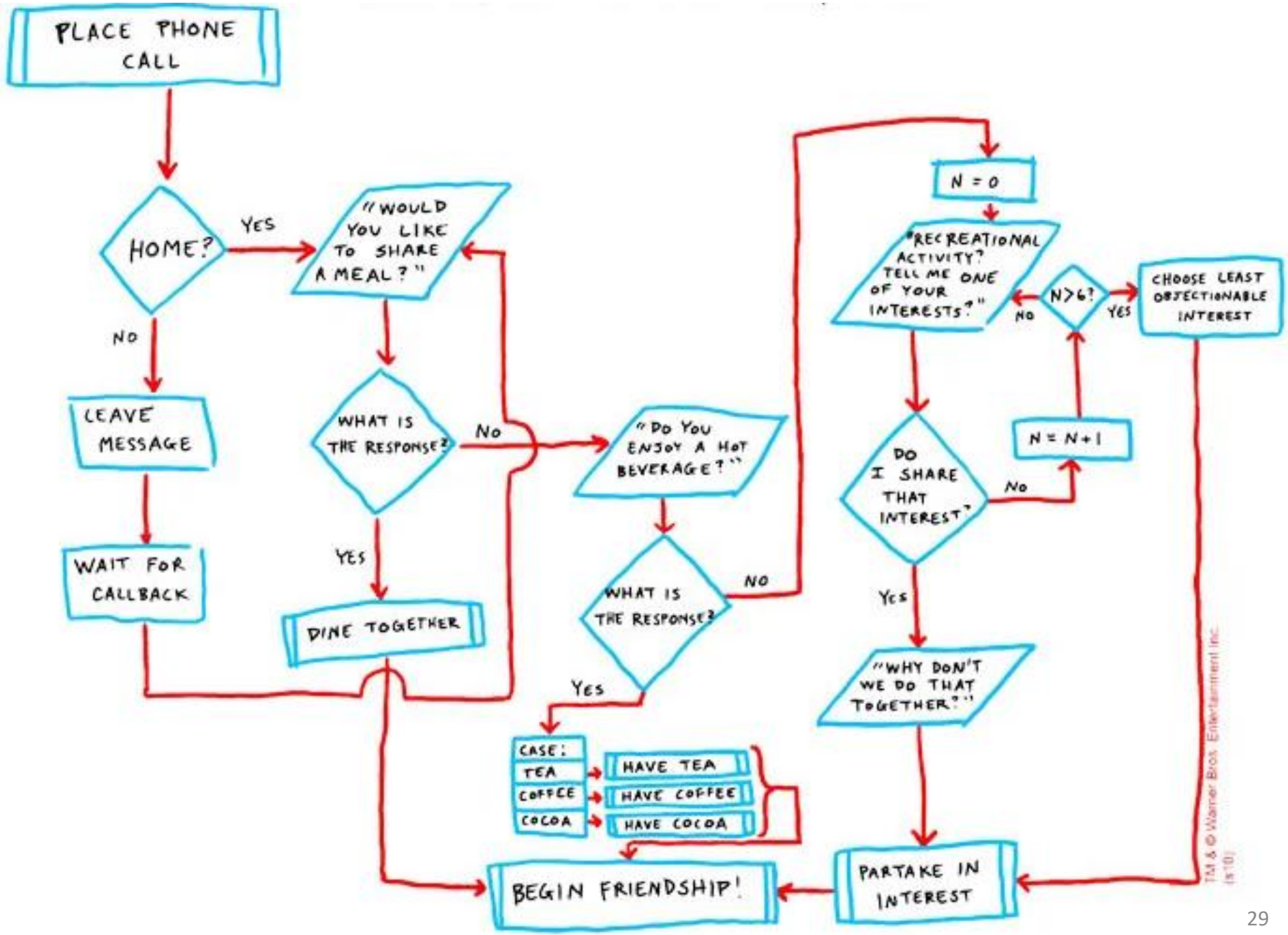


I would like to make new friends…



THE FRIENDSHIP ALGORITHM

DR. SHELDON COOPER, Ph.D

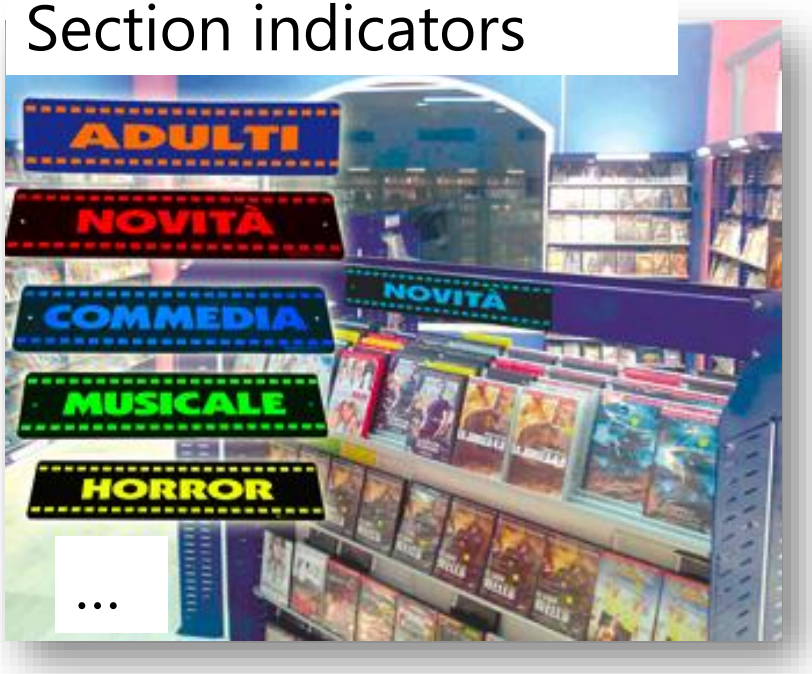


Buy a DVD

Where do I search for it?
In which *section*?

Action

Section indicators



After I found the section, how do I find the DVD?

Alphabetic order

Algorithm for buying a DVD

1. **Acquire** the name of the section (s) containing the DVD (d)
2. Go to the section (s)
3. **Search**
4. Get the DVD (d)

- **Acquire** and **Search** are also *algorithms*

SEARCH: context and problem

- Context
  - ➢You are in the correct section (s)
  - ➢In the section there are several DVDs
    - The section can be seen as a set of DVDs: $s=\{d_0, d_1, \ldots, d_n\}$

- Problem
  - ➢I have to find the DVD (d_t) among all DVDs in the section (s)
  - ➢**Help**: the section is sorted
    - DVDs are sorted alphabetically

SEARCH algorithm

- Known
  - $s=\{d_0, d_1, \ldots, d_n\}$, sorted alphabetically
  - DVD searched = d_t

1. Is there any DVD?

- **Then**
  1. **Read** the title of the first DVD in s
  2. **If** the title is the searched title
     Then end the search successfully
     Else move to the next DVD and restart from 1
- **Else** end the search with a negative outcome

SEARCH: remarks

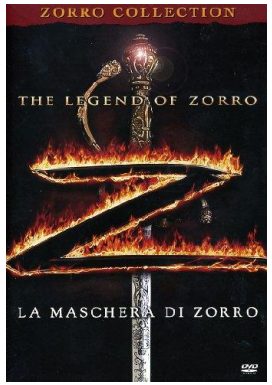
- How does my search *work*?
  - If in s there are 10000 DVDs and I search for “Zorro”…?
  - Even worse scenario
    - You want to search for the Italian home address of your professor in the address book of the whole Italy…
- There are several ways to solve a problem

Known:

d_t

$s=\{d_0, d_1, \ldots, d_{11}\}$

d_0	d_1	d_2	d_3	d_4	d_5	d_6	d_7	d_8	d_9	d_{10}	d_{11}



Read d_x and compare to d_t

$d_x = d_6$

$d_6 = d_t$?

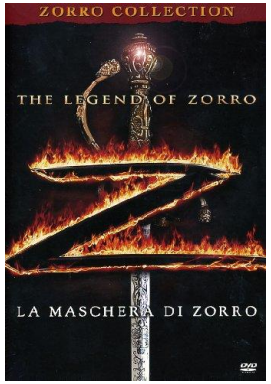
NO

$d_6 < d_t$

Restart *on the upper half*

d_t

d_6	d_7	d_8	d_9	d_{10}	d_{11}



$d_x = d_9$

Read d_x and compare to d_t

$d_x = d_9$

d_9 = d_t ?

YES

End the search successfully buying d_9

SEARCH… improved

- Known
  - ➢ $s=\{d_0, d_1, \ldots, d_n\}$, sorted alphabetically
  - ➢ DVD searched = d_t

1. Is there any DVD?

- **Then**
  1. **Read** the title of the DVD (d_x) in the *middle* of s
  2. **If** the title of d_x is the searched title
     - **Then** end the search successfully
     - **Else if** $d_x < d_t$
       - **Then** restart from 1 considering the second half of s
       - **Else** restart from 1 considering the first half of s
- **Else** end the search with a negative outcome

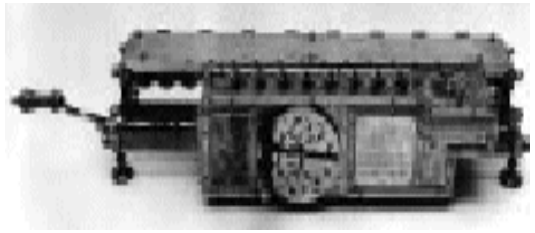
WELCOME TO THE FANTASTIC WORLD OF CODING!

BUT FIRST...

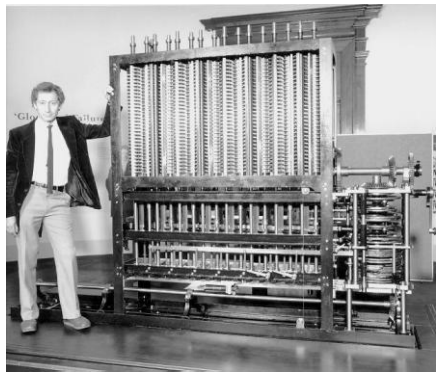
A little bit of history…

- Blaise Pascal (1623-1662)
  mechanical device (gears controlled by a crank) for executing addition and subtraction

- Gottfried Wilhelm von Leibniz (1646-1716)
  introduces also multiplication and division (four-function calculator)



- Charles Babbage (1792-1871)
  designs and creates a "difference engine"
  - Calculates tables of numbers useful for navigation
  - only one algorithm: finite-difference polynomial
  - output: holes on a copper plate (punch cards)



A little bit of history…

- Programming machine: analytical engine (C. Babbage)
  - Made of four parts:
    - **store** (memory: 1000 cells × 50 digits)
    - **mill** (computational unit: 4 operations + data transfer)
    - **input** (plate reader)
    - **output** (plate hole-puncher)
  - Too advanced for the technology of that time: too many hardware errors (precision cogwheels)

- The role of programmer was born:
  - Ada Augusta Lovelace

A little bit of history…

- **COLOSSUS** (England 1943)
  - The first digital electronic calculator
  - Used for decoding secret German messages
  - Military secret for 30 years, therefore irrelevant

- **ENIAC** (Mauchley and Eckert - USA 1946)
  - Electronic Numerical Integrator And Computer
  - Made of 18000 fuses e 1500 relays for an overall weight of 30 tons and a consumption of 140 kw

A little bit of history…

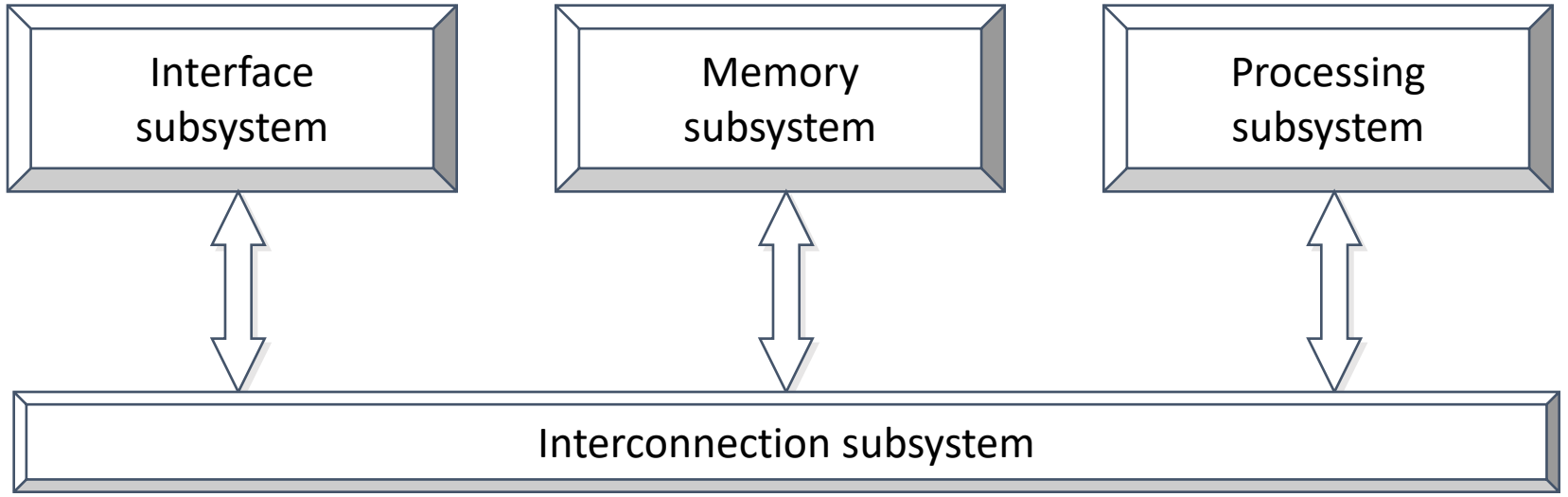
John von Neumann

- Participated to the ENIAC project
- His project (von Neumann machine) is still nowadays the base of almost all digital calculators

Von Neumann architecture

A calculator must be able to:

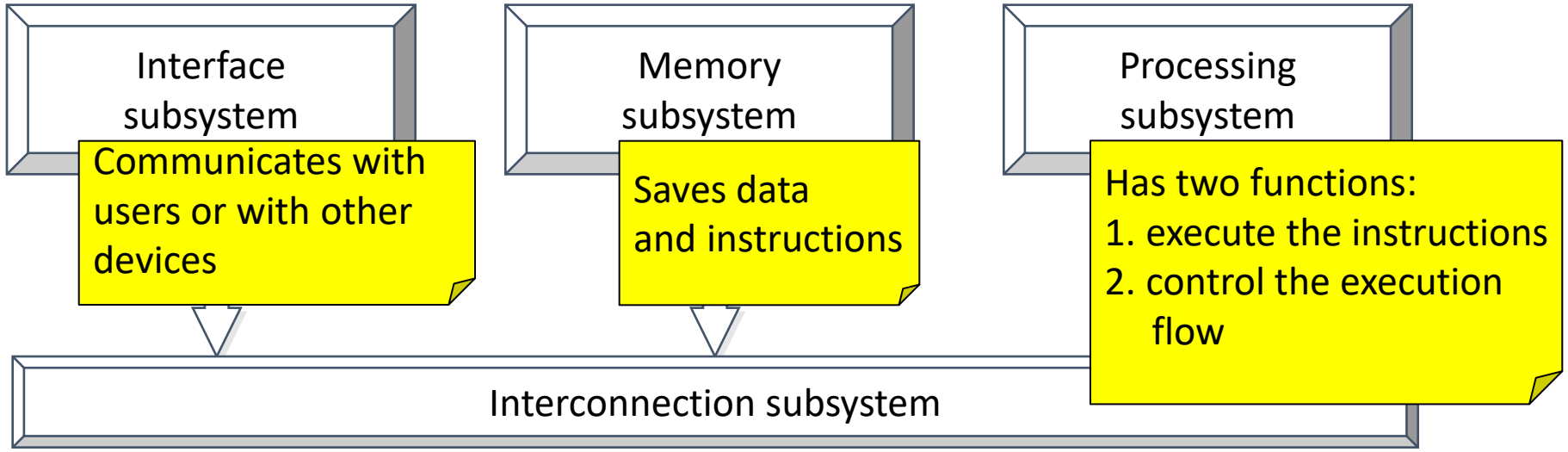
- execute instructions on data
- control the execution flow
- save the data to process
- save sequences of instructions
- interact with users and with other potential systems



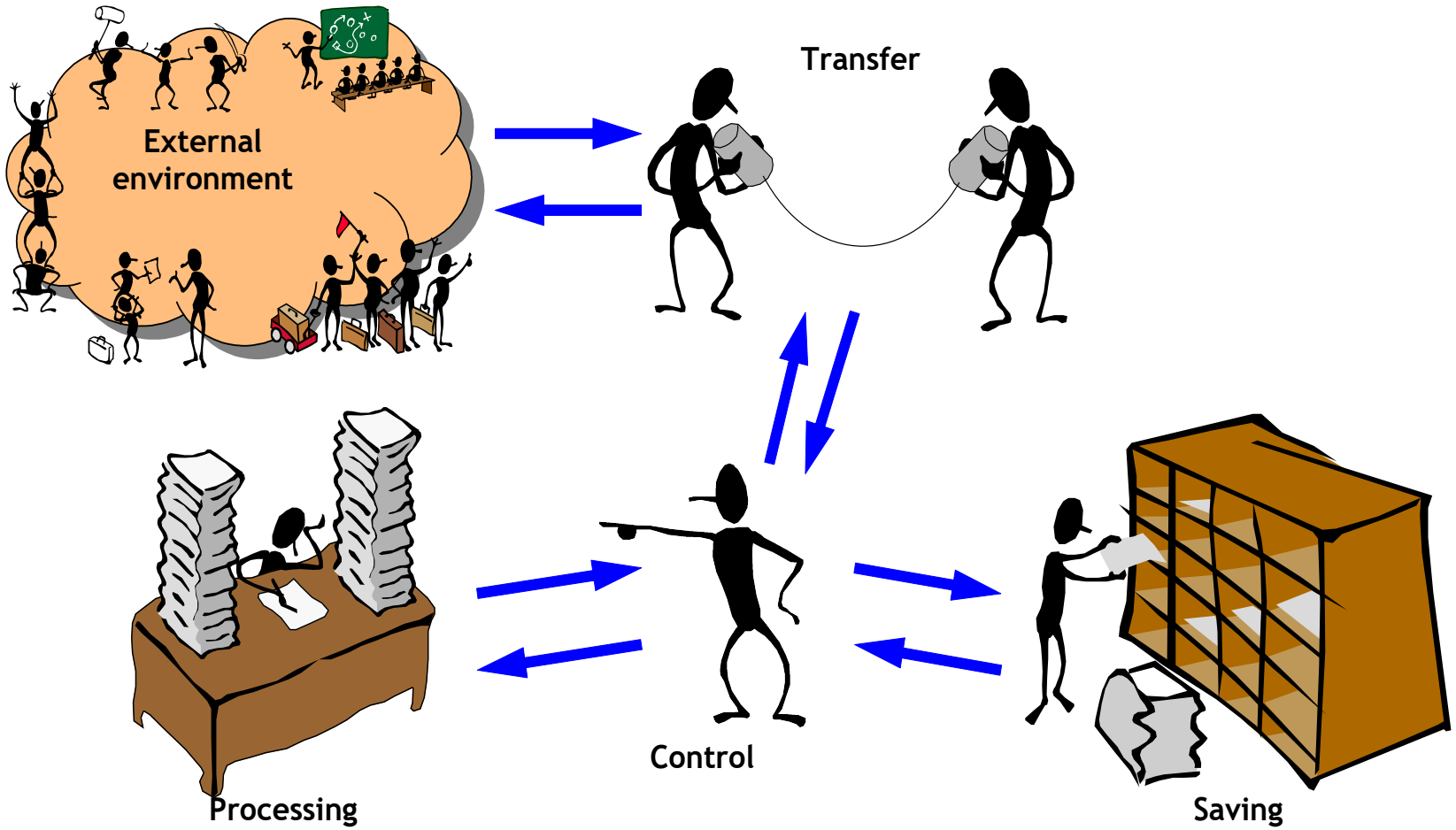
Von Neumann architecture

A calculator must be able to:

- execute instructions on data
- control the execution flow
- save the data to process
- save sequences of instructions
- interact with users and with other potential systems



Functionalities of a calculator



WELCOME TO THE FANTASTIC WORLD OF CODING!

Encoding data and instructions

Algorithm

- Description of the solution of a problem, written in such a way to be able to be executed by an executor (potentially different from the author of the algorithm).
- Sequence of instructions that operate on data.

Program

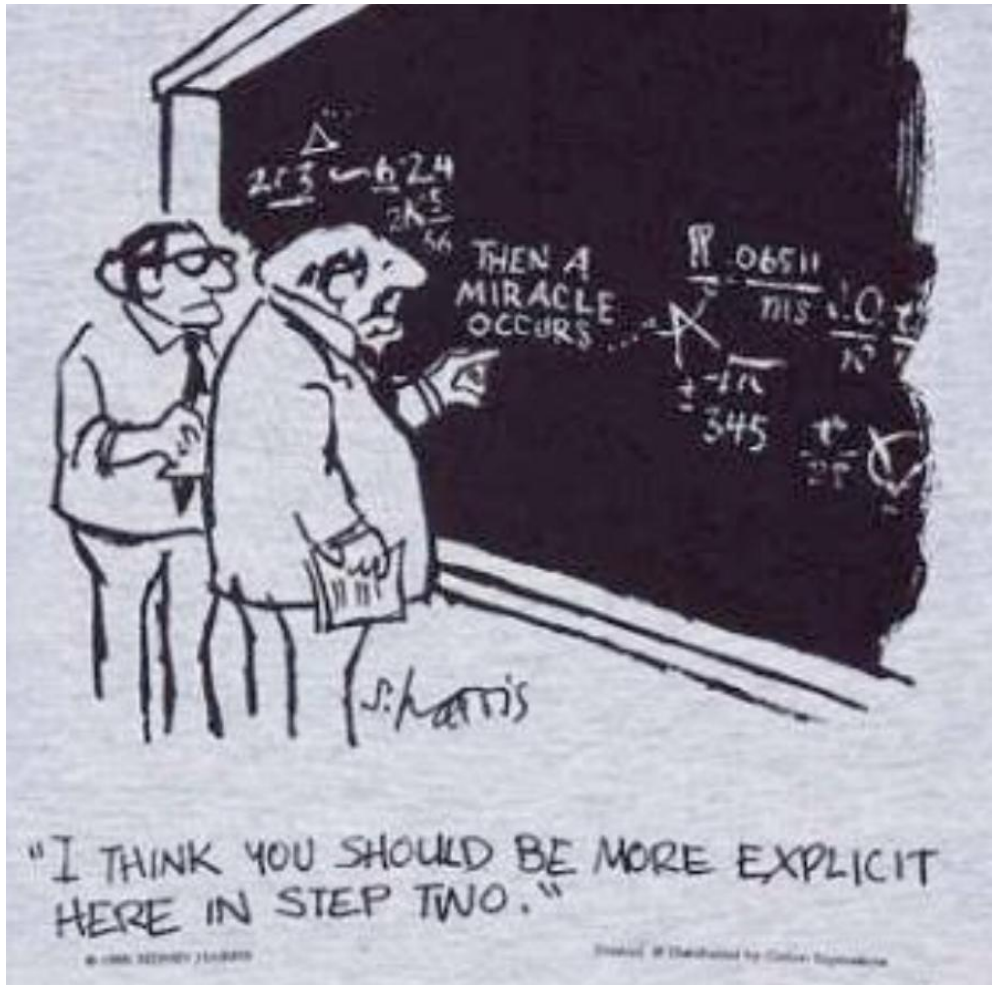
- Algorithm written in such a way to be able to be executed by a calculator (automatic executor)

Process

- Running program

Encoding data and instructions

To write a program, it is necessary to represent instructions and data in a format that the automatic executor is able to save and handle



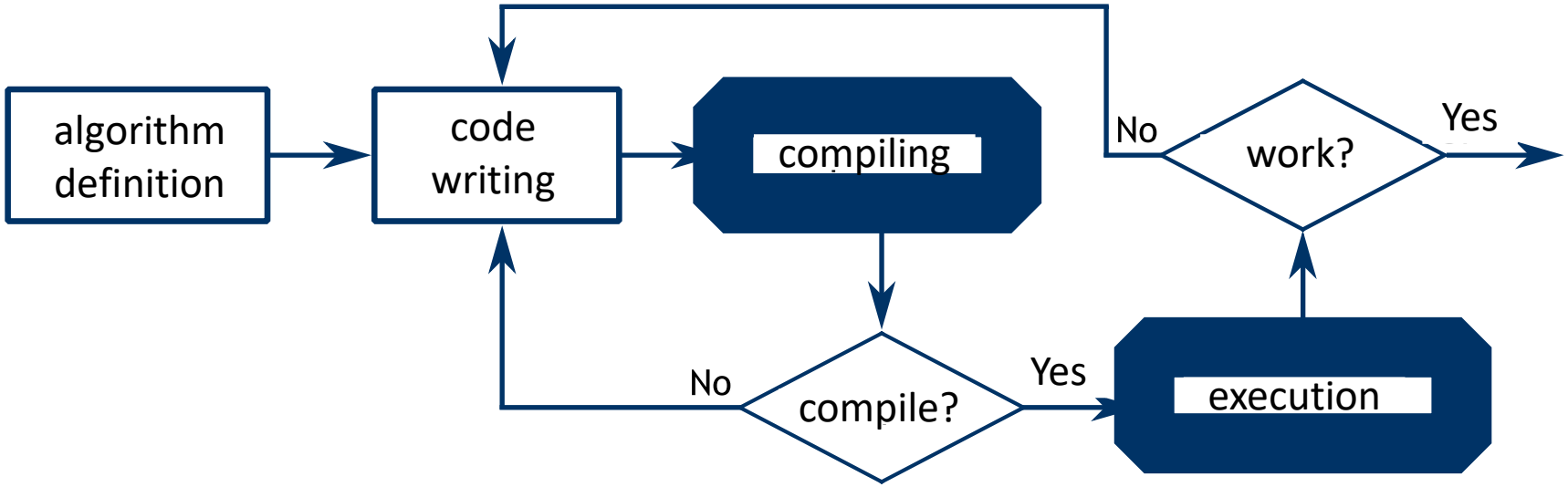
The “executor”

- A calculator based on von Neumann architecture executes a program on the basis of the following principles:
  - data and instructions are saved in a single memory that allows both writing and reading;
  - the contents in the memory are indexed on the basis of their position, regardless the type of data or instruction inside;
  - the instructions are executed in a sequential manner.

- The language that allows the CPU to behave as an executor is called *machine language*.

How to compile and execute

- To be able to execute our program, we need to translate it into machine language
  - This operation is executed by an interpreter



JUPYTER NOTEBOOK